

NetCompose: Interactive Network Topology Design and Automated Container-Based Deployment

José Lopes

Faculdade de Engenharia da Universidade do Porto
Email: up202208288@up.pt

Pedro Marinho

Faculdade de Engenharia da Universidade do Porto
Email: up202206854@up.pt

Abstract—Configuring experimental networks requires detailed knowledge of IP addressing, routing, service deployment, DNS, and container orchestration. These tasks are repetitive and error-prone when performed manually, especially when a user wants to quickly test different network designs. This paper presents NetCompose, a web-based tool that allows users to visually design network topologies and automatically generate a runnable Docker-based emulation environment. The system supports hosts, switches, routers, servers, load balancers, DNS servers, static routing, and OSPF-based dynamic routing. A React-based frontend exports a declarative topology model, while a FastAPI backend generates Docker Compose files, per-device initialization scripts, and service configuration files. The evaluation validates a representative service architecture and proposes a benchmark methodology for comparing manual, AI-assisted, and NetCompose-based deployment. The preliminary evaluation suggests that, for the tested topology, NetCompose reduced setup time and manual configuration effort compared with manual and AI-assisted approaches.

I. INTRODUCTION

Network laboratories are commonly used to test routing behavior, service placement, load balancing, name resolution, and failure scenarios. However, building even a modest topology manually requires a sequence of low-level configuration steps: creating isolated networks, assigning IP addresses, configuring default gateways, enabling IP forwarding, writing static routes or OSPF configuration, deploying services, and validating connectivity. In Docker-based environments this also includes writing Compose files, creating build contexts, and ensuring that containers have the required Linux networking capabilities.

This process is useful pedagogically, but it becomes slow and error-prone when the goal is to rapidly prototype and compare alternative designs. A single typo in a subnet, route, or gateway can make the topology fail in ways that are difficult to debug. In addition, when services such as load balancers and DNS servers are added, the user must keep application-level configuration consistent with the network-level configuration.

Real network incidents also show that configuration mistakes can cascade across layers. For example, Meta’s October 2021 outage started with backbone router configuration changes that disrupted data-center communication and caused DNS routes to be withdrawn, making Facebook, Instagram, and WhatsApp unreachable [8]. Incidents of this type motivated NetCompose: by allowing users to build and run network scenarios in a controlled environment, the tool helps them

observe how routing, DNS, and service placement interact before applying similar changes to real systems.

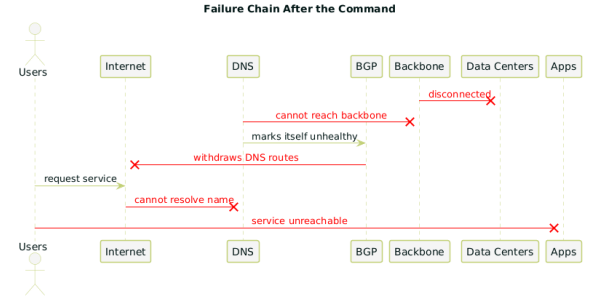


Fig. 1: Cascading failure during the October 2021 Meta outage, illustrating how backbone router misconfigurations withdrew DNS routes.

NetCompose addresses this problem by providing an interactive visual interface for creating network topologies and a backend generator that converts the visual model into an executable Docker Compose environment. The goal is not to hide networking concepts completely. Instead, the tool exposes important parameters, such as IP addresses, masks, gateways, OSPF areas and costs, DNS records, and service domains, while automating the repetitive steps needed to instantiate the topology.

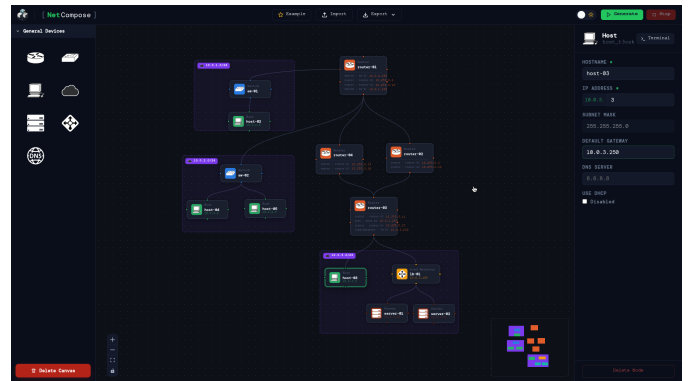


Fig. 2: The NetCompose interactive web interface, demonstrating the visual topology canvas and the device configuration sidebar.

The main contributions of this project are:

- a visual topology editor that exports a declarative JSON representation of networks, devices, links, and layout;
- a backend generator that creates Docker Compose services, networks, build contexts, and initialization scripts;
- support for static routing and OSPF routing using FR-Routing, including route convergence after path failures;
- support for application-level services such as HTTP servers, an Nginx load balancer, and a DNS server with automatically generated and user-editable records;
- an interactive command endpoint and terminal UI for testing connectivity and inspecting container state.

II. RELATED WORK

Several existing tools can be used to create or manage virtualized network environments. Docker Compose is a simple and widely used tool for defining multi-container applications, but it provides only a textual configuration interface and leaves most network and service configuration to the user. GNS3 and EVE-NG provide powerful network emulation environments, but they are heavier tools and often target more specialized router images and network appliances. Cisco Packet Tracer provides a beginner-friendly visual simulator for networking concepts, but it does not deploy real Docker services such as DNS servers, Nginx load balancers, or custom HTTP servers.

Mininet is another relevant tool for network experimentation, especially in the context of software-defined networking. It allows users to create virtual hosts, switches, links, and controllers on a single machine. However, Mininet is more focused on programmable network emulation and SDN experiments than on visually generating Docker Compose laboratories with application-level services such as DNS, HTTP servers, and load balancers.

Infrastructure-as-code tools such as Ansible and Terraform can automate configuration and deployment, but they require users to write configuration files and understand the corresponding abstractions. Kubernetes provides scalable orchestration for production systems, but its complexity is excessive for small laboratory topologies.

Compared with these tools, NetCompose focuses on a narrower educational and prototyping use case: visually designing a topology and automatically generating a runnable Docker-based lab. The objective is to combine the accessibility of a visual simulator with the practical value of real container-based services.

III. SOLUTION

A. Overview

NetCompose is composed of a frontend topology editor, a backend generator, and a Docker-based execution environment. Figure 3 shows the high-level architecture of the system.

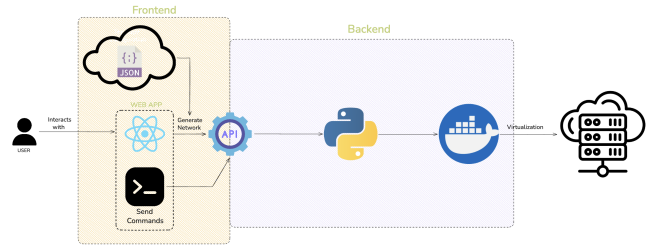


Fig. 3: High-level architecture of NetCompose.

The frontend provides a sidebar with the available devices. Users can drag devices to the canvas, place them inside networks, connect them, and edit their configuration. The resulting topology is exported as JSON and sent to the backend, which generates the Docker Compose environment required to run the lab.

B. Topology Model

The exported topology relies on four main components: networks, devices, links, and layout data. Networks establish the required subnets and gateways, while the devices store their own type-specific configurations. These elements are tied together by links that map out the logical connections across the system. Additionally, the model includes layout data, such as node positions and sizes, so that the exact visual arrangement is preserved if a topology is later re-imported.

Using this declarative format keeps the backend completely decoupled from React Flow’s internal state. The backend does not need to track where the user placed the nodes on the canvas; it simply uses the logical relationships between the networks, devices, and links to generate the deployment files.

C. Device Types

NetCompose currently supports the following device types:

- **Host:** endpoint container used for connectivity testing and command execution.
- **Switch:** lightweight device representing Layer 2 connectivity inside a network.
- **Router:** multi-interface device with IP forwarding, static routing, and optional OSPF support.
- **Server:** HTTP endpoint used to test service access and load balancing.
- **Load balancer:** Nginx-based reverse proxy that forwards requests to connected server nodes.
- **DNS server:** dnsmasq-based name server with automatically generated and manually editable records.

D. Routing

The tool supports two routing modes. When OSPF is disabled, the backend computes static routes from the topology graph and installs them in router initialization scripts. The topology is treated as a graph where routers and networks are connected through links and interfaces. For each router, the backend uses a breadth-first search (BFS) traversal to discover

the reachable remote networks and determine the next hop needed to reach them. It then generates the corresponding Linux static route commands, associating each destination subnet with the appropriate next-hop router interface. This mode is deterministic and works well for simple labs.

The current prototype uses single-area OSPF by default, with interfaces assigned to area 0.0.0.0. However, the model stores OSPF information at the interface level, which makes future support for multi-area OSPF possible.

By using OSPF, the system supports failure experiments. For example, in a diamond topology with two paths between two LANs, OSPF selects the lower-cost path. If a router on that path stops, OSPF reconverges, and traffic can move to the backup path after a short convergence delay. This behavior demonstrates that the generated topology is not only a static container deployment but also a useful network experimentation environment.

E. DNS and Load Balancing

The server, load balancer, and DNS components extend the project beyond basic connectivity. Servers run a small HTTP service that identifies the responding backend. The load balancer generates an Nginx upstream from connected server nodes and exposes a domain such as `app.local`. The DNS server generates records from server and load balancer domains, while still allowing the user to add manual records through the UI.

If a host has a configured DNS server, its initialization script overwrites `/etc/resolv.conf` to use that server. Unknown records are forwarded by the DNS server to an upstream resolver such as 8.8.8.8. This allows both internal names, such as `app.local`, and external names to be resolved.

F. Implementation

The frontend is implemented in React and React Flow. It manages node creation, visual layout, automatic hostname generation, subnet assignment, router interface generation, DNS record synchronization, and JSON import/export. The backend is implemented in FastAPI and Python. It exposes endpoints to run and stop the topology, execute commands inside containers, and return command output to the UI.

Docker Compose is used as the deployment target. The backend converts the topology model into the files required to start the lab, including the Compose file and the initialization scripts for each device. This keeps the deployment process simple while still allowing the generated configuration to be inspected and modified if needed.

IV. EVALUATION

The evaluation was divided into two parts. First, we validate that NetCompose can correctly generate and execute a representative network architecture. Second, we define a benchmark methodology to compare the effort required to create the same environment using NetCompose and using alternative approaches, such as manual configuration or AI-assisted generation.

A. Architecture Validation

To validate the system, we used the topology shown in Figure 4. The architecture contains two client hosts, one router, one load balancer, and three backend servers. The goal of this scenario is to verify that NetCompose can generate a complete runnable environment where clients can reach a service through a load balancer, while the underlying network configuration is generated automatically.

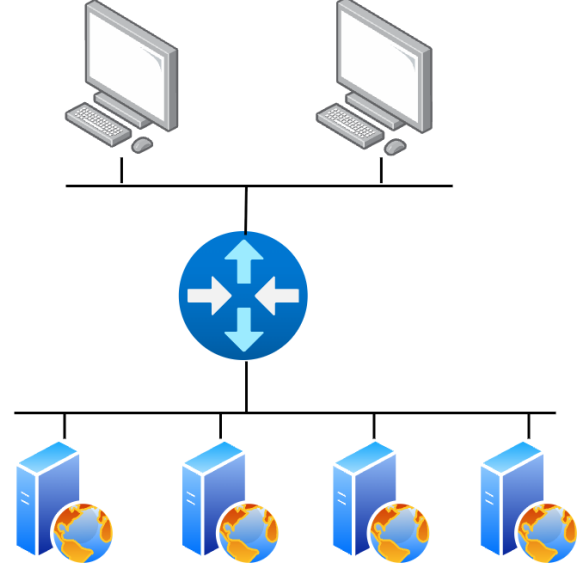


Fig. 4: Evaluation topology with two clients, one router, one load balancer, and three backend servers.

This topology represents a common service deployment pattern: users access an application through a single load-balancer address, while requests are distributed across multiple backend servers. In a manual setup, this requires the user to create Docker networks, assign IP addresses, configure gateways, write startup scripts, define the Nginx upstream configuration, and verify connectivity between all components. NetCompose reduces this process to the creation of a visual topology and the generation of the corresponding Docker-based environment.

The following tests were used:

- **Container deployment:** all generated containers must start successfully.
- **IP configuration:** each device must receive the IP address and gateway defined by the topology.
- **Client connectivity:** client hosts must be able to reach the router and the load balancer.
- **Backend connectivity:** the load balancer must be able to reach each backend server.
- **HTTP service:** clients must be able to access the application through the load balancer.
- **Load balancing:** repeated HTTP requests should be distributed across the backend servers.

Table I summarizes the validation results.

TABLE I: Architecture validation tests.

Test	Expected	Status
Containers start	All running	YES
Client ping	Router/LB reachable	YES
Backend ping	Servers reachable	YES
HTTP request	Response received	YES
Repeated requests	Multiple servers used	YES

This validation is important because the objective of NetCompose is not only to draw a topology, but to generate a working network laboratory. The user does not need to directly interact with Docker Compose, Linux routing commands, or Nginx configuration files. Instead, the system generates these artifacts from the topology model, allowing users with limited Docker experience to test network and service behavior more easily.

B. Benchmark Methodology

The benchmark compares the time required to reproduce the validation topology using three approaches: manual configuration, AI-assisted generation, and NetCompose. The measured time corresponds to the time needed to obtain a runnable deployment, including corrections when the first result was not functional.

This benchmark is not intended as a general performance evaluation. A complete usability study would require a larger group of users with different levels of networking, Docker, and AI-assistant experience, which is outside the scope of this course project. Therefore, we keep the evaluation limited to a small controlled comparison using two or three participants familiar with Docker, Docker Compose, networking concepts, and AI-assisted configuration.

For NetCompose, the value is the average time of three users who had not previously used the application. The measured task started with creating the topology in the UI and ended when the generated environment was running. For the AI-assisted approach, the value is the average of three AI assistants. The prompt used for each AI assistant is included in Appendix A. The AI-assisted time includes both prompt preparation and the time required to obtain runnable files, making the comparison closer to a realistic user workflow. Claude did not generate a correct version on the first attempt, but produced a working version on the second attempt. The manual setup was tested by two users familiar with Docker, Docker Compose, and Linux networking.

TABLE II: Setup time comparison for the validation topology.

Approach	Individual Times	Avg. Time
Manual setup	12:10, 13:10	12:40
AI-assisted setup	1:55, 2:05, 2:24	2:08
NetCompose	1:22, 1:34, 1:46	1:34

The results show that NetCompose was the fastest approach for this topology. More importantly, it allowed users with no previous experience with the tool to create and run the network without manually writing Docker Compose, Nginx, or Linux

networking configuration files. The AI-assisted approach was also faster than manual setup, but it still required reviewing and correcting generated files. Manual setup was the slowest because all configuration artifacts had to be written and checked by hand.

TABLE III: Configuration artifacts required for the validation topology.

Artifact	Manual	NetCompose
Compose file	Written manually	Generated
Dockerfiles	Written manually	Generated
Startup scripts	Written manually	Generated
IP/gateway config	Written manually	Generated/editable
Nginx config	Written manually	Generated
Server config	Written manually	Generated
Topology layout	Not reusable	Exported/imported

This second comparison highlights the main benefit of the system: NetCompose does not only reduce setup time, but also reduces the number of low-level configuration files the user needs to understand and maintain.

V. DISCUSSION

NetCompose shows that a visual topology model can be used not only as a diagram, but also as the source of a runnable network laboratory. This is the main advantage of the approach: the same model that the user edits visually is used to generate Docker Compose files, startup scripts, routing configuration, DNS records, and service configuration. As a result, the risk of inconsistencies between the diagram and the deployed environment is reduced.

The tool is also useful from an educational perspective. Instead of hiding networking concepts, it exposes important parameters such as IP addresses, gateways, OSPF costs, DNS records, and service domains. Users can therefore experiment with these values and immediately observe their effect in real containers. This makes the system more than a deployment shortcut; it also acts as a learning environment for understanding how routing, name resolution, and service reachability depend on each other.

The main limitation is that the current backend targets Docker Compose. This makes the prototype easy to run and suitable for small laboratories, but it does not provide the same realism as a full network emulator or a production orchestration platform. Docker bridge networks also introduce behavior that may differ from physical networks, such as default gateways and simplified Layer 2 behavior. Therefore, NetCompose should be understood as a lightweight experimentation and learning tool rather than a complete replacement for professional network emulators.

A. Future Improvements

Several improvements could extend NetCompose beyond its current proof-of-concept stage. The most important future direction is the development of a topology analyzer capable of inspecting the graph before deployment and reporting warnings or errors.

A second improvement is the addition of more components, such as firewalls, monitoring probes, NAT gateways, VPN endpoints, or additional routing protocols.

Another useful extension is support for executing user-provided files inside selected containers. For example, a user could upload or select a benchmark script and run it on a host, server, or router through the UI. This would make it possible to automate experiments such as latency measurements, throughput tests, repeated DNS lookups, HTTP load-balancing tests, or OSPF convergence measurements. The results could then be collected by the backend and displayed in the frontend as tables or charts.

Finally, the platform could be improved with readiness checks and observability features. After starting a topology, the backend could automatically verify whether containers are running, gateways are reachable, OSPF adjacencies are established, DNS records resolve correctly, and load balancers can reach their backend servers. These checks would turn NetCompose from a deployment generator into a more complete network management tool.

VI. CONCLUSION

This paper presented NetCompose, a visual tool for designing and deploying Docker-based network laboratories. The system converts a topology drawn in a web UI into a Docker Compose environment with hosts, routers, switches, servers, load balancers, DNS servers, static routes, and OSPF. The project demonstrates that a visual topology model can reduce repetitive configuration work while preserving user control over important networking parameters.

Developing the prototype also deepened our practical understanding of IP addressing, routing, DNS, OSPF, and service reachability. Making the network actually run revealed practical issues that are easy to miss when studying only the theory, such as gateway selection, routing convergence, DNS reachability, and consistency between service configuration and network configuration.

The prototype can already be used as a visual learning lab for beginners. With more devices, protocols, validation mechanisms, and deployment targets, it could evolve into a more realistic network emulator, allowing advanced users to use it to validate changes and test failure scenarios before deploying them in real networks.

REFERENCES

- [1] Docker Inc., “Docker documentation.” [Online]. Available: <https://docs.docker.com/>
- [2] Docker Inc., “Docker Compose documentation.” [Online]. Available: <https://docs.docker.com/compose/>
- [3] xyflow, “React Flow documentation.” [Online]. Available: <https://reactflow.dev/>
- [4] S. Ramírez, “FastAPI documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [5] FRRouting Project, “FRRouting documentation.” [Online]. Available: <https://docs.frrouting.org/>
- [6] F5, Inc., “NGINX documentation.” [Online]. Available: <https://docs.nginx.com/>
- [7] S. Kelley, “dnsmasq.” [Online]. Available: <https://thekelleys.org.uk/dnsmasq/doc.html>
- [8] Meta Engineering, “More details about the October 4 outage.” [Online]. Available: <https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/>

APPENDIX A

PROMPT USED FOR AI-ASSISTED BENCHMARK

The following prompt was used to evaluate the AI-assisted setup approach. The prompt describes the same validation topology used in the NetCompose benchmark and asks the model to generate all required Docker Compose and configuration files.

Create a complete Docker Compose-based network lab with the following topology:

- Two client hosts: client-01 and client-02
- One router connecting the client network to the server network
- One Nginx load balancer
- Three backend HTTP servers: server-01, server-02, and server-03

Requirements:

1. Use Docker Compose.
2. Create two isolated Docker networks:
 - client_net: 10.0.1.0/24
 - server_net: 10.0.2.0/24
3. The router must connect to both networks:
 - client_net IP: 10.0.1.250
 - server_net IP: 10.0.2.250
4. client-01 and client-02 must be in client_net.
5. The load balancer and the three backend servers must be in server_net.
6. Assign static IP addresses to all containers.
7. Configure each client with the correct default gateway so it can reach the server network through the router.
8. Configure the router with IP forwarding enabled.
9. Configure the Nginx load balancer to listen on port 80 and forward traffic to the three backend servers.
10. Each backend server should run a simple HTTP service that returns its own name, so it is possible to verify load balancing.
11. Include all necessary Dockerfiles, init scripts, Nginx configuration files, and Docker Compose configuration.
12. The final result should be runnable with docker compose up --build.
13. Also provide commands to test:
 - client connectivity to the router
 - client connectivity to the load balancer
 - HTTP access from a client to the load balancer
 - repeated HTTP requests showing that traffic reaches multiple backend servers

Return the complete file structure and the full contents of every file.